Phase 1 Report: Exploration and Preprocessing of the COCO Data

Mini-project: image captioning system using RAG (MS COCO)

Master 1, Intelligent Information Systems Engineering

Group no. [to be completed] | Members: [to be completed]

November 11, 2026

Groupe 24: Sadeki Nadjlaa,Laib Fatima, Serine Baghdadi, Hebbache ibtihal remaki

1. Introduction

This report presents our work on the first phase of the image captioning mini-project. The goal of the project is to build a system that receives an image and produces a sentence describing it, for example "a man is playing tennis". For this we use the MS COCO dataset, which contains images together with captions written by humans.

A model that generates captions on its own has two weaknesses: it sometimes forgets objects that are in the image, and it invents others that are not there. This second weakness is called hallucination, and it is one of the important metrics of the project. To reduce it, we use RAG (Retrieval Augmented Generation). For a given image, we first look in a knowledge base for similar images and captions, we add these examples to the model's prompt, and then the model writes the caption using them.

Phase 1 prepares this knowledge base. We had to explore the data, clean it, split it into three sets (knowledge base, validation and test), compute the embeddings and build a search index. This report follows that order. All the work was done with a Python script in four steps (splits, stats, embed, index), run locally in VS Code, with a fixed random seed (seed = 42) so that the results can be reproduced exactly.

2. Presentation of the dataset

MS COCO (Common Objects in Context) is a collection of images of everyday scenes. We used the 2017 version. The train2017 folder contains 118,287 images and the val2017 folder contains 5,000. The annotations are stored in JSON files: for each image there are about 5 captions written by different people (591,753 captions for train2017) and the list of objects present with their class. There are 80 object classes (person, car, chair, dog, etc.).

File	Content	Role in our work
captions_train2017.json	118,287 images and 591,753 captions	knowledge base and validation set
instances_train2017.json	annotated objects, 80 classes	object statistics
captions_val2017.json	captions of the val2017 images	test set
instances_val2017.json	annotated objects of val2017	choice of test images by complexity

The project statement mentions a subset of about 10,000 images. Since we downloaded the full dataset, we randomly drew 10,000 images from train2017 ourselves to build the knowledge base (see section 5).

3. Audit and cleaning

Before analysing the data, we ran three checks.

1. **Missing images.** Every image listed in the JSON file must exist in the folder and have at least 5 captions. Only the images that pass this test can be drawn. For train2017, all 118,287 images of the JSON file were present in the folder with at least 5 captions.
2. **Corrupted images.** Each selected image is opened with the PIL library (verify method). If it cannot be opened, the image is discarded.
3. **Duplicates.** We compute the MD5 fingerprint of each file. Two files with the same fingerprint are identical, and we keep only one of them.

When an image is discarded, it is replaced by another image drawn from the rest of the dataset, so that each set keeps exactly the wanted size. All discarded images are recorded in the file audit_problemes.json.

Result: this file is empty. No corrupted image and no duplicate was detected among the selected images (10,000 for the knowledge base, 1,000 for validation and 1,000 for test). It should be said that the check covered these images and not all 118,287 images of the folder, which was enough since only these are used afterwards.

For the captions, the knowledge base contains 50,024 captions for 10,000 images. This is slightly more than 5 per image because a few COCO images have more than 5 captions (24 extra captions in total). We kept all the captions.

4. Statistical exploration

All the statistics in this section are computed on the knowledge base (10,000 images and 50,024 captions). The objects come from the file instances_train2017.json.

4.1 Object classes

The 10,000 images contain objects from all 80 COCO classes, about 73,357 annotated objects in total (7.34 objects per image on average). Figure 1 shows the 50 most frequent classes.

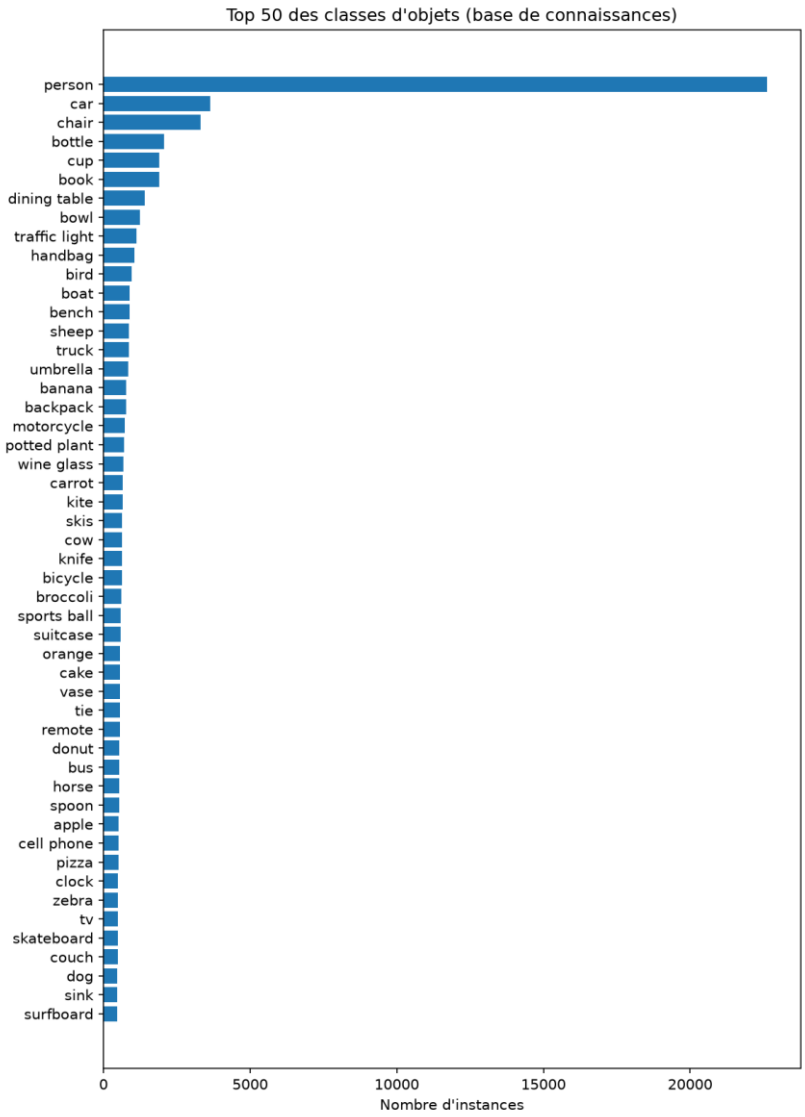


Figure 1: the 50 most frequent object classes in the knowledge base.

The class person clearly dominates with 22,631 instances, which is about 31% of all objects. It is followed by car (3,647), chair (3,315), bottle (2,060), cup (1,911), book (1,898), dining table (1,396), bowl (1,247), traffic light (1,116) and handbag (1,051). From the tenth class on, we are below 1,100 instances, and the last classes of the top 50 (couch, dog, sink, surfboard) have between 465 and 490 instances.

The distribution is therefore very unbalanced: one huge class, a few medium classes, and then a long tail of rare classes. For the rest of the project, this means that the examples retrieved by the RAG will often talk about people, and that rare classes will be less represented in the base.

4.2 Scene complexity

We measured the complexity of a scene in two ways: by the number of annotated objects in the image, and by the number of different classes. On average, an image contains 7.34 objects and 2.92 different classes. Figure 2 gives the two histograms.

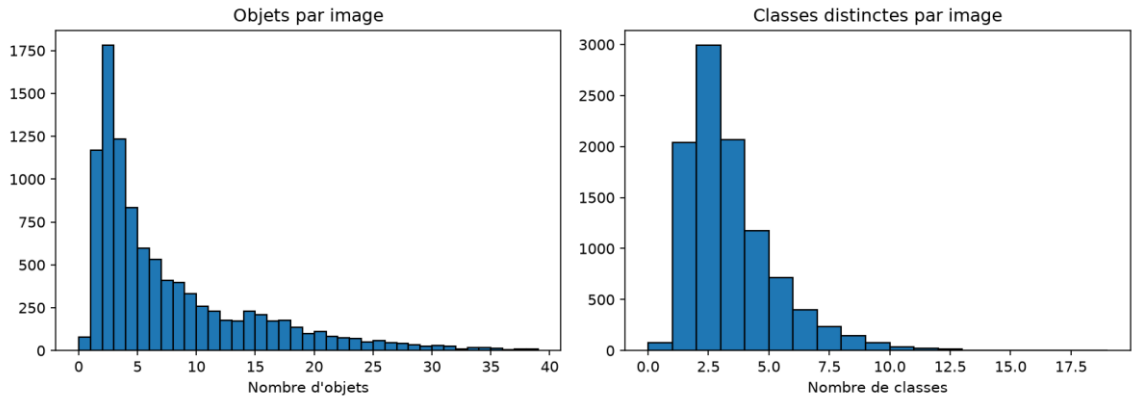


Figure 2: number of objects per image (left) and number of distinct classes per image (right).

On the left histogram, the distribution is stretched to the right. The peak is at 2 objects (about 1,800 images), then the number of images decreases slowly, with a tail that goes beyond 35 objects. The mean of 7.34 is therefore pulled up by a few very crowded scenes (crowds, streets, store shelves). Most images have few objects. About 80 images have no annotated object.

On the right histogram, most images have between 1 and 4 different classes, with a peak at 2 classes (about 3,000 images). An image can therefore contain many objects, but they are often of the same class (for example several people).

This criterion was also used to choose the test set: we split the val2017 images into simple, medium and complex scenes according to their number of objects (see section 5).

4.3 Captions: length and vocabulary

The base contains 50,024 captions. To count words, we put the captions in lowercase and kept only letters and apostrophes. The average length is 10.47 words. Figure 3 shows that most captions have between 8 and 12 words, with a peak around 9 to 10 words (about 10,800 captions for each of these two lengths). Almost no caption has fewer than 7 words, and a few longer captions go beyond 20 words.

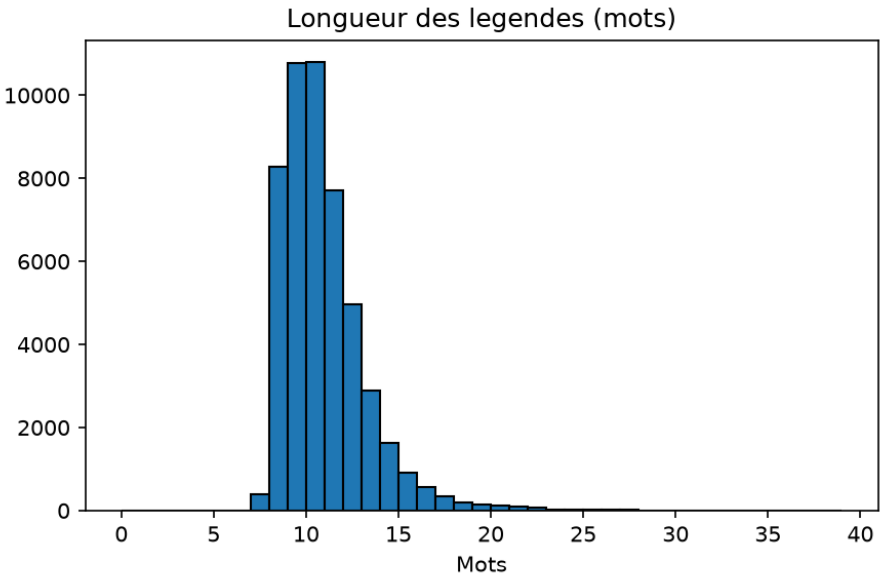


Figure 3: distribution of caption length, in number of words.

The vocabulary contains 9,732 different words, which is quite small compared with the 50,024 captions. The annotators therefore use short sentences with simple, repeated words. For the RAG this is convenient: the retrieved captions will be short, so we can put several of them in a prompt without making it too long.

4.4 Parts of speech (POS)

The project statement also asks for the distribution of grammatical categories (nouns, verbs, adjectives, etc.). This part of the script uses the nltk library. During our run it could not be completed: the file stats.json contains an empty value for the POS distribution. We therefore have no result to present here for the moment.

4.5 Co-occurrence of objects

To see which objects appear together, we built a co-occurrence matrix on the 20 most frequent classes (Figure 4). Each cell shows the number of images that contain both classes at the same time. Since the values are very different, the colour represents log(1 + number of images). The diagonal gives the number of images that contain each class.

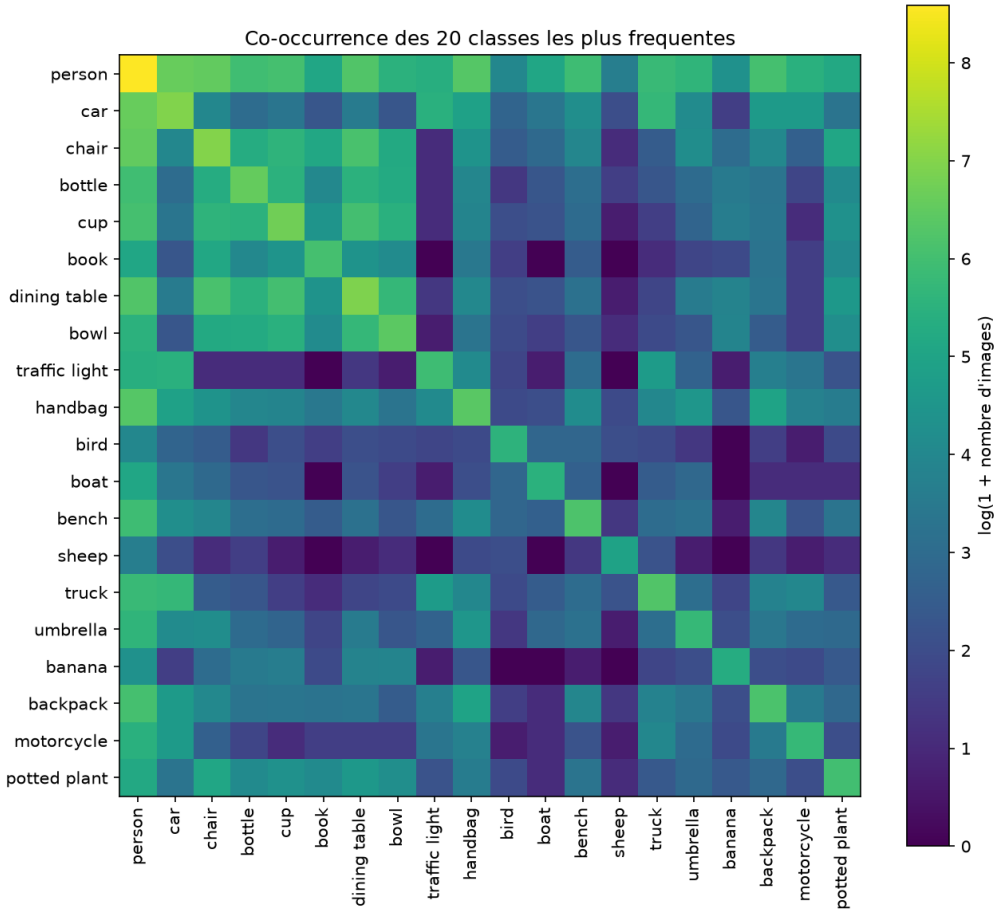


Figure 4: co-occurrence matrix of the 20 most frequent classes.

Several things are clearly visible:

- **person** is yellow on the diagonal, which means it is present in about half of the images (in the order of 5,000). Its row is light almost everywhere: we find it with car, chair, dining table, handbag, backpack, etc.
- Indoor objects form a group: chair, dining table, cup, bowl and bottle often appear together, which matches kitchen or dining room scenes.
- Street objects form another group: car, truck and traffic light.
- Handbag and backpack often appear together.
- Some pairs almost never meet: book with boat, sheep or traffic light, banana with bird, boat or sheep, sheep with traffic light.

These groupings show that COCO contains quite typical scenes. A model can take advantage of this, but it can also add "usual" objects that are not in the image (for example a table when it sees chairs). This is a possible source of hallucinations that we will monitor in phase 2.

4.6 Colours and contexts

To measure the diversity of the scenes, we searched for keywords in the captions. The percentages below are the share of the 50,024 captions that contain the word.

Colours (Figure 5). White is the most cited colour (5.8%), then black (3.2%), red (2.9%), blue (2.3%) and green (2.1%). Yellow and brown are at 1.4%, orange at 0.8%, pink at 0.6%, and gray and purple at 0.3%. Colours are therefore cited in few captions: the annotators mostly describe objects and actions. Two limits should be mentioned. We counted only "gray" and not "grey", so gray is underestimated. And "orange" can also refer to the fruit.

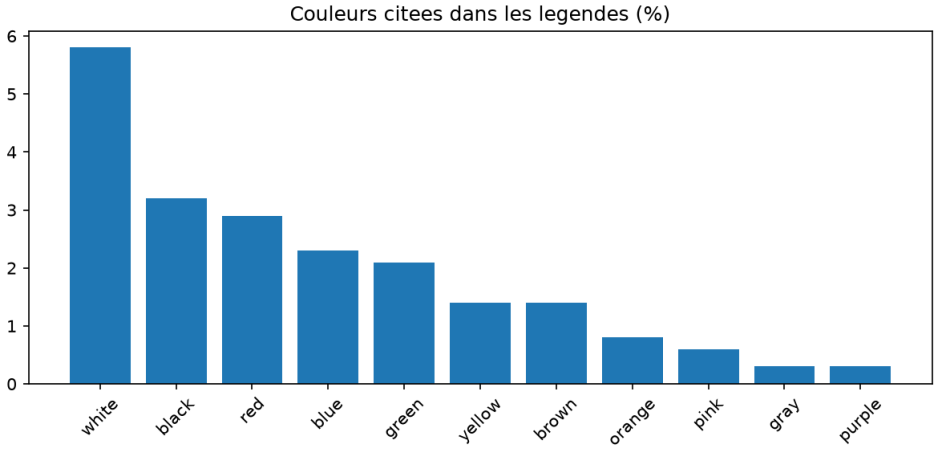


Figure 5: share of captions that mention each colour (in %).

Contexts. With word lists (kitchen, room, bathroom, bedroom, table, indoor, inside, office for indoor; street, field, beach, park, outdoor, road, grass, snow, mountain, water for outdoor; night, dark, evening for night), we obtain 10.9% of indoor captions, 12.4% of outdoor captions and only 0.5% of night scenes. The first two figures are close, so the base mixes indoor and outdoor scenes well. Night scenes are very rare. This method remains approximate: it is not a real classification of the images, and most captions contain none of these words, so the percentages do not add up to 100%.

5. Splitting into three sets

We created three separate sets. The project statement gives different sizes depending on the passage (about 100 or 1,000 images for the test); we kept 1,000 images for validation and 1,000 for test.

Set	Images	Source	Role
Knowledge base	10,000	train2017, random draw	retrieves similar examples (RAG)
Validation	1,000	train2017, outside the base	tunes parameters during development
Test	1,000	val2017	final evaluation, untouched before

How we proceeded.

1. We keep the valid images of train2017 and shuffle them with seed 42. The first 10,000 form the base and the next 1,000 form the validation set. The validation set is therefore outside the base: if a validation image were in the base, it would retrieve its own caption and the results would be wrong.
2. The test set comes from val2017, a folder completely separate from train2017. We sorted its images by number of objects and cut them into three groups: simple, medium and complex scenes. We then draw in each group (334, 333 and 333 images) so that the test set contains all difficulty levels.
3. The identifiers of the three sets are saved in three files (kb_ids.json, val_ids.json, test_ids.json), so the sets never change again.

Leakage check. The script checks that no image is in two sets at the same time. The result is saved in rapport_splits.json: 0 leakage and 0 audit problem. In addition, the indexed captions are only those of the 10,000 images of the base, so no validation or test caption can be found by the search.

6. Preprocessing

Images. Before computing the embeddings, each image is opened, corrected according to its EXIF orientation (some photos are saved rotated) and converted to RGB, so that all of them have the same colour format, including the rare grayscale images. The resizing is then done by the CLIP model itself, which puts the images at the size it expects.

Captions. We keep two versions of each caption:

- the original version (captions_raw), which never changes and will be used as the reference to compute metrics such as BLEU;
- the cleaned version (captions_clean), which is only used for indexing. It is lowercased, punctuation is removed (we keep letters, digits and apostrophes) and extra spaces are removed. For example, "A man riding a wave." becomes "a man riding a wave".

Keeping the original is important: if we evaluated generated captions against cleaned captions, the scores would be slightly changed. Both versions are saved in the metadata folder, together with the list of objects of each image.

7. Embeddings

An embedding is a vector of numbers that represents the meaning of an image or a text. Two images that look alike have close vectors, and this is what allows us to retrieve similar examples. We used two models:

- **CLIP (clip-ViT-B-32)** for the images and for the captions. CLIP places images and texts in the same space, so we can search captions starting from an image.
- **Sentence-BERT (all-MiniLM-L6-v2)** for the captions only, to compare text with text.

All vectors are normalized (norm 1), so cosine similarity equals the dot product. They are saved in .npy format.

Embeddings	Model	Vectors	Dimension	Time (s)
Base images	CLIP ViT-B/32	10,000	512	1,002.2
Validation images	CLIP ViT-B/32	1,000	512	65.4
Test images	CLIP ViT-B/32	1,000	512	68.3
Base captions	CLIP (text encoder)	50,024	512	452.5
Base captions	Sentence-BERT	50,024	384	127.1

The computation was done on CPU, without a GPU, and took about 29 minutes in total (1,715.5 seconds). Images are the slowest part, about 0.1 second per image, whereas a caption takes about 9 ms with CLIP and 2.5 ms with Sentence-BERT.

8. Vector index

To quickly search for the closest vectors, we used FAISS. We created three indexes, one for each type of embedding of the base. All are of type IndexFlatIP: the search is exact, meaning the query is compared with all the vectors, and the measure is the dot product. Since the vectors are normalized, this is the same as cosine similarity. With at most 50,024 vectors, an exact search remains very fast, so we did not need an approximate index.

Index	Type	Metric	Dim.	Vectors	Size (MB)	Indexing time (s)
kb_img_clip	IndexFlatIP	cosine	512	10,000	20.5	0.006
kb_cap_clip	IndexFlatIP	cosine	512	50,024	102.4	0.029
kb_cap_sbert	IndexFlatIP	cosine	384	50,024	76.8	0.020

Building each index takes less than a tenth of a second, which is negligible compared with the computation of the embeddings.

Quality check. Before saving the report, the script checks that there are no missing values (NaN) in the embeddings, that the vectors are properly normalized, that there is no duplicated identifier, that no validation or test image or caption is in the base, and that the number of vectors matches the number of images and captions. It also runs a search test with the validation and test images: the returned results do come from the base. The file rapport_index.json is only written if all these checks pass.

An image of the base necessarily retrieves itself as its first result. In the phase 2 pipeline, it will therefore have to be excluded from the search when testing on images of the base.

9. Difficulties and limits

- **Working environment.** We first tried on Kaggle, but the dataset is very heavy and the notebook restarted, losing the variables of the cells already run. We moved to a local script, divided into four steps that save their results in files. If a step stops, it can be run again without redoing everything.
- **Computation time.** Without a GPU, the embeddings took about 29 minutes. This is not blocking for 10,000 images, but it would be long with a larger base.
- **POS analysis.** It could not be completed during our run (see 4.4).
- **Colours and contexts.** They are measured with keyword lists, so the figures are approximate and rather underestimated.
- **Audit.** It covers the images kept in the three sets, not the whole dataset.
- **Drawing of the base.** The 10,000 images were drawn by our group with seed 42. If the other groups drew different images, the comparison of results between groups will be less direct. We will have to check with the teacher whether a common list of images is planned.
- **Size of the sets.** The project statement is not consistent about the number of test images, which also needs to be confirmed with the teacher.

10. Conclusion and next steps

This first phase allowed us to prepare a clean knowledge base: 10,000 images and 50,024 captions, with no corrupted image and no duplicate, with a validation set (1,000 images) and a test set (1,000 images) separated from the base and without leakage. The statistics taught us several things: the class person is very dominant, most scenes are simple, captions are short (10.5 words on average) with a small vocabulary, and some objects always appear together, which can encourage hallucinations.

The files produced are stored in the phase1_output folder:

Folder	Content
splits	identifiers of the three sets, splits report, audit report
metadata	raw and cleaned captions, objects of each image
stats	graphs and statistics (stats.json)
embeddings	.npy vectors and embeddings report
index	FAISS indexes and index report

Phase 2 will consist of building the RAG pipeline: retrieving similar examples from our indexes (image to image, image to caption, with CLIP or Sentence-BERT, and other strategies such as BM25 for comparison), adding these examples to the prompt, generating the caption, and then evaluating with the BLEU score, the F1 score, the hallucination rate and the execution time. We will use the validation set to tune the parameters, and the test set only for the final evaluation.